UNIVERSITÀ
DEGLI STUDI
FIRENZE

Frozen Transformers in Language Models Are Effective Visual Encoder Layers

Ziqi Pang Ziyang Xie Yunze Man Yu-Xiong Wang

ICLR 2024

Cosimo Borghini

Obiettivo

Mostrare che i transformers degli LLM addestrati **solo su testo** sono dei layer efficienti per i visual **encoders** anche in **assenza di ulteriori dati testuali**.

Proposta

Estrarre un blocco di transformer **congelato** da un LLM e aggiungerlo a un visual encoder.

Da dove nasce l'idea?

- I LLM mostrano potenzialità anche su task **non**-linguistici.
- Ciò è noto nei modelli **multi-modali**, in cui per esempio:
  - Si proiettano embeddings visuali nello spazio di embeddings testuali.
  - Cross-Attention tra token visuali e quelli testuali.

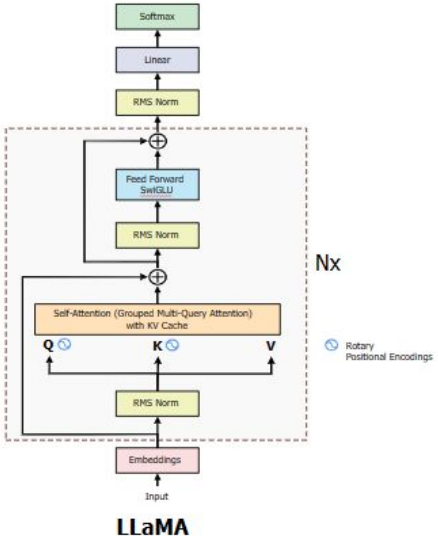
    ➔ **Utilizzano dati visuali e testuali**.

Può un transformer pre-addestrato, risolvere **task visuali** in assenza di **dati testuali?**

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Come?

Come? (Intuitivo)

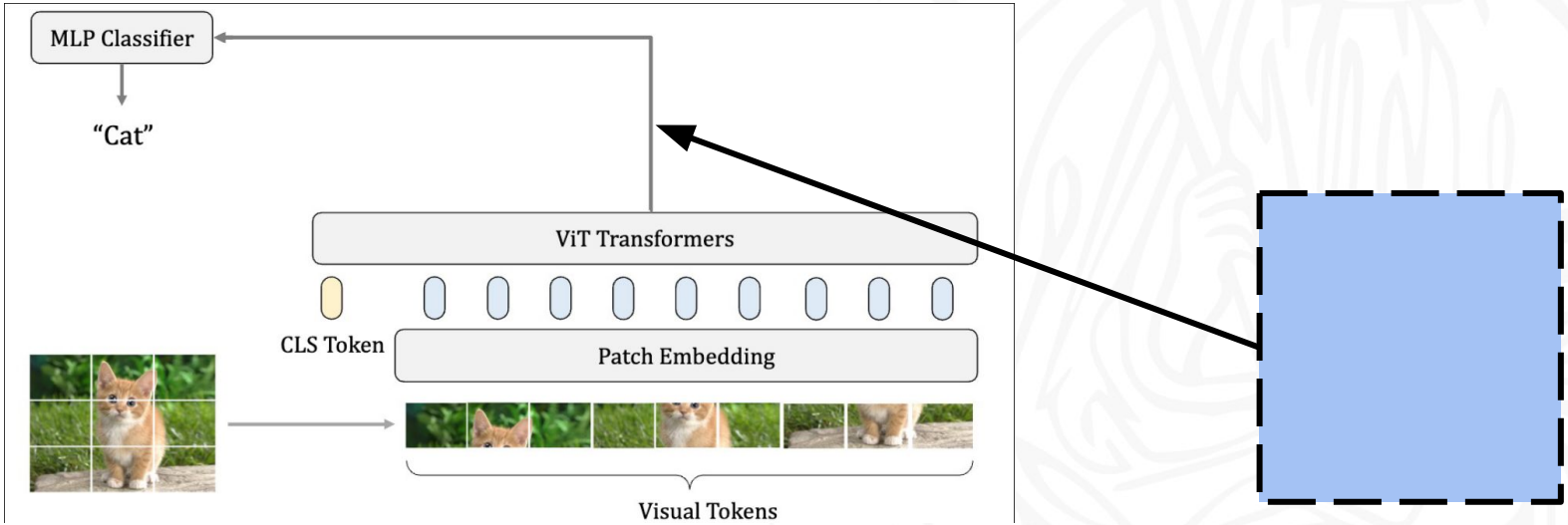


Come?

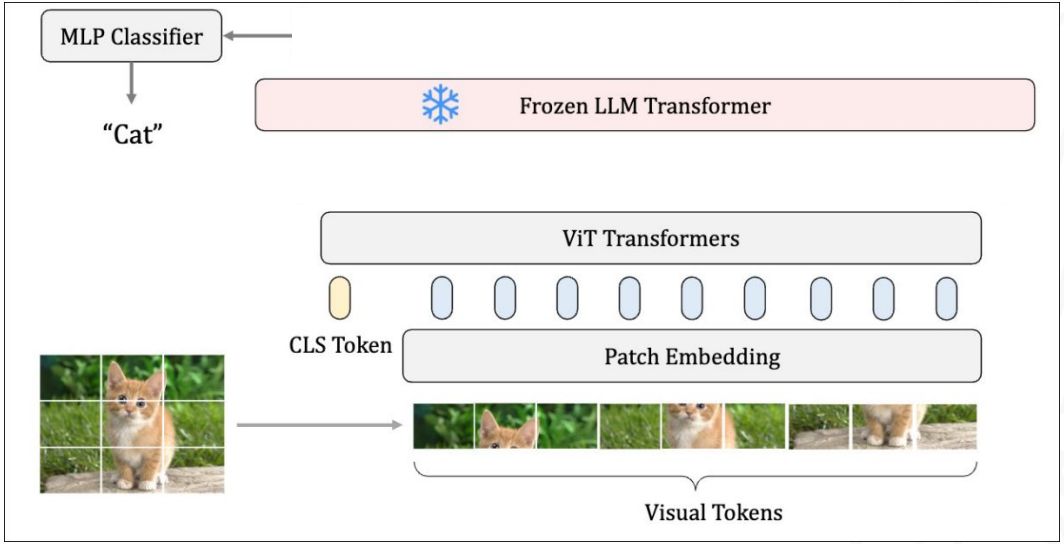
1. Si estrae un blocco transformer congelato da un LLM.

Come?(Caso Classificazione Immagini)

1. Si estrae un blocco transformer congelato da un LLM e lo si colloca in testa a un visual encoder.

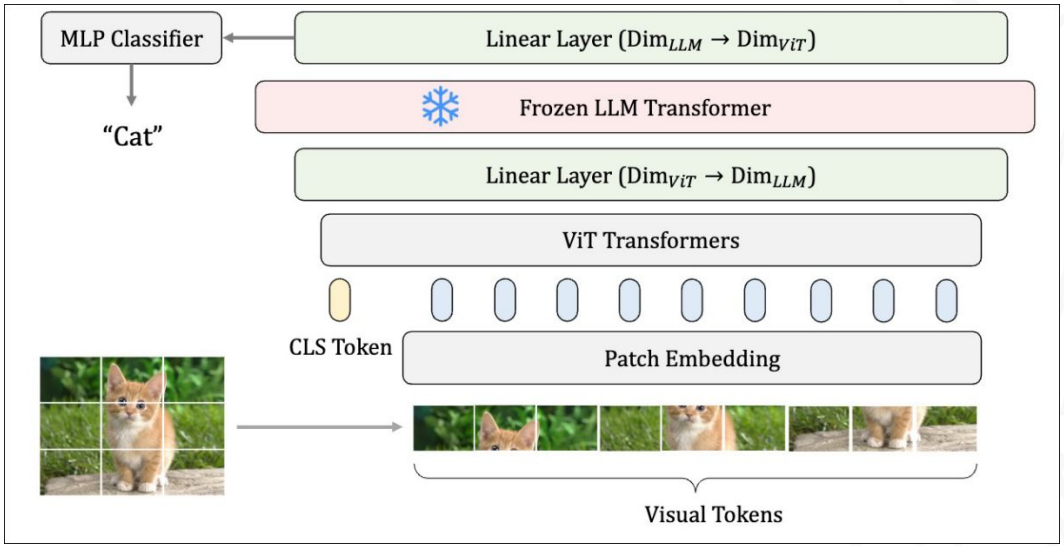


Come?


MLP Classifier
"Cat"
Frozen LLM Transformer
ViT Transformers
CLS Token
Patch Embedding
Visual Tokens

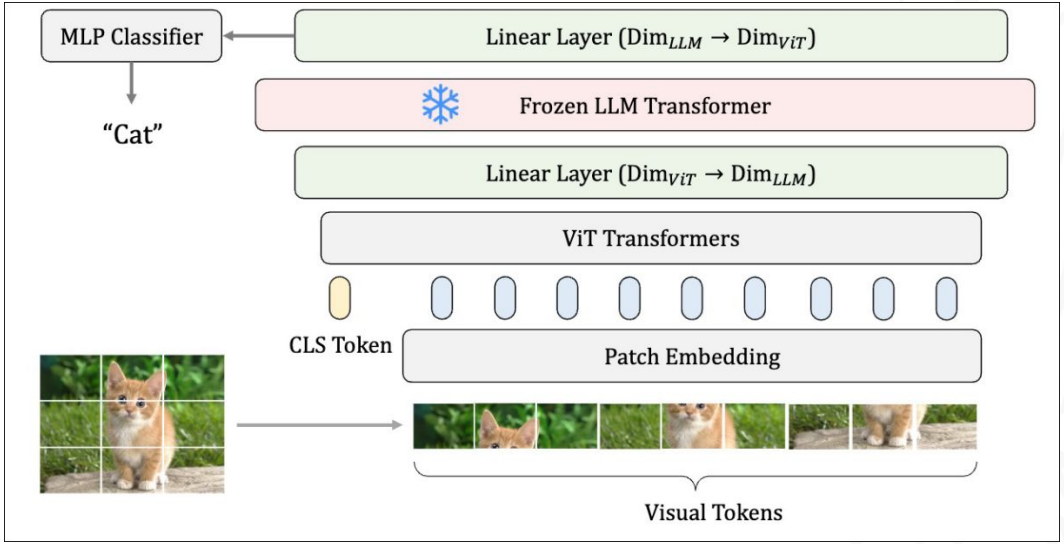
Come?

2. Si aggiungono due layer addestrabili per allineare le dimensioni dello spazio delle feature.



Come?

3. Si mantiene il transformer congelato durante l'addestramento.




```python
def __init__(self, *args, **kawargs):
    # Encoder
    self.ViT = Encoder(args, kwargs)
    self.classifier = Decoder(args, kwargs)

    # Language Transformer
    self.L1 = nn.Linear(ViT.hidden_dim, LM.hidden_dim)
    self.LM = LM_Transformer(args, kwargs)
    self.L2 = nn.Linear(LM.hidden_dim, ViT.hidden_dim)

    # Freezing
    for param in self.LM.parameters():
        param.requires_grad = False

def forward(self, img):
    z = self.ViT(x)

    z = self.L1(z)
    z = self.LM(z)
    z = self.L2(z)

    y = self.classifier(z)
    return y
```

Come? (Formale)

Si considera una rete neurale che mappa l'**input x** in nella **rappresentazione z** e predice l'**etichetta y**, utilizzando un **encoder** ed un **decoder**.

$$\mathbf{F}_E(x) \to z, \quad \mathbf{F}_D(z) \to y.$$

Come?

Si aggiunge poi un **singolo blocco transformer pre-addestrato** proveniente da un LLM.

$$\mathbf{F}_E(x) \to z, \qquad \mathbf{F}_{LM} \quad (z) \to z', \quad \mathbf{F}_D(z') \to y.$$

Come?

Siccome le **feature** hanno **dimensioni differenti** tra **l'encoder** e il **transformer,**
si utilizzano **due linear layers** prima e dopo il transformer.

$$\mathbf{F}_E(x) \to z, \quad \mathbf{F}_L^2 \cdot \mathbf{F}_{LM} \cdot \mathbf{F}_L^1(z) \to z', \quad \mathbf{F}_D(z') \to y.$$

Modifiche al Transformer:

1. No auto-regressive masking:

   Non serve sequenzialità temporale.
   Si usa il masking solo per i PAD token.

2. No positional embeddings:

   Rimossi embeddings come RoPE in LLaMa per coerenza con visual encoder.

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Perché è interessante?

Perché questo approccio è interessante?

0. Migliora performance dei modelli baseline.

Perché questo approccio è interessante?

0. Migliora performance dei modelli baseline.
1. Il modello non riceve input testuali.

Perché questo approccio è interessante?

0. Migliora performance dei modelli baseline.
1. Il modello non riceve input testuali.
2. No backbone con pre-training su immagini, si addestra da 0.

Perché questo approccio è interessante?

0. Migliora performance dei modelli baseline.
1. Il modello non riceve input testuali.
2. No backbone con pre-training su immagini, si addestra da 0.
3. LLM non più “monolitico” ma come collezione di blocchi riutilizzabili.

Risultati: Classificazione Immagini

Model	ImageNet	ImageNet-C	ImageNet-A	ImageNet-SK	ImageNet-R
ViT-T	72.1	43.9	7.7	19.6	32.3
+LLaMA	**73.2**	**45.8**	**8.7**	**20.6**	**33.8**
ViT-S	80.1	57.2	20.5	28.9	42.1
+LLaMA	**80.7**	**58.7**	**22.7**	**30.5**	**42.8**
ViT-B	80.6	60.5	23.4	31.9	43.5
+LLaMA	**81.7**	**62.1**	**26.9**	**33.2**	**44.3**

Risultati: Point Cloud Classification (3D)

Model	ScanObjectNN		
	BG	OBJ	T50
Point-BERT	87.4	88.1	83.1
+LLaMA	**88.0**	**88.5**	**83.8**

Model	ModelNet40		
	1k	4k	8k
Point-BERT	**92.67**	**92.91**	93.19
+LLaMA	92.42	92.82	**93.56**

Risultati: Action Recognition (Video)

Model	Acc1	Acc5
ViT-S	64.71	89.15
+LLaMA	**65.89**	**89.93**
ViT-B	64.97	89.50
+LLaMA	**66.03**	**90.25**

Risultati: Motion Forecasting (Non-Semantic)

Model	ADE↓	FDE↓	MR↓
VectorNet	0.77	1.23	13.2
+LLaMA	**0.76**	**1.20**	**12.7**
mmTransformer	0.72	1.10	10.7
+LLaMA	**0.71**	**1.08**	**10.5**

Studio delle scelte di Design

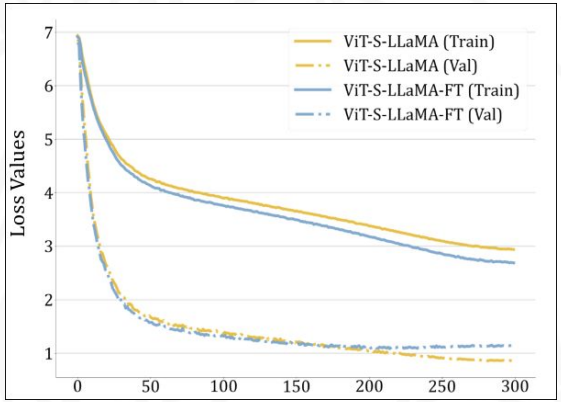
Scelte di design: Capacità del Modello

- Creando **ViT-S-MLP** con lo stesso numero di parametri di ViT-S-LLaMA ma senza LLM, si ottiene un miglioramento parziale.
- Risultato: **solo metà del guadagno di ViT-S-LLaMA**, dimostrando che **i pesi pre-addestrati del LLM sono cruciali**, non è solo questione di maggior capacità.

Model	Acc
ViT-S	80.1
ViT-S-LLaMA	**80.7**
ViT-S-MLP	80.4
ViT-S-LLaMA-FT	78.9

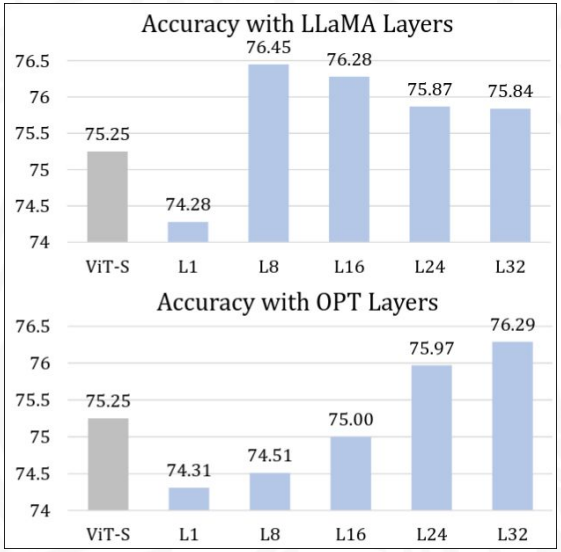
Scelte di design: Fine-tuning e Congelamento

- Fine-tuning del LLM (ViT-S-LLaMA-FT) **peggiora le prestazioni** rispetto al modello congelato.
- Motivazione: overfitting, training loss basso ma validation loss alto.
- Conclusione: **congelare il LLM è più semplice ed efficace**.



Scelte di design: Scelta del layer del LLM

- Diversi layer del LLM (OPT o LLaMA) hanno **impatto significativo** sulle feature visive, anche con capacità identica.
- I **layer finali** tendono a migliorare le prestazioni, sebbene non sempre siano ottimali.
- Implicazione: **la selezione del layer del LLM è importante** e la metodologia funziona con vari LLM.



Perché funziona?

Information Filtering Hypotesis

"Il blocco transformer pre-addestrato del LLM agisce come un **filtro** che identifica i **token informativi** e ne **amplifica il contributo** alla predizione, aumentando la **magnitudine** o le **componenti frequenziali** delle attivazioni delle feature."

Filtro:

Sono stati predisposti i metodi per il calcolo e la visualizzazione delle magnitudini e delle frequenze.

```
def activations(visual_tokens):
    # visual_tokens: tensor with shape [H, W, C]
    # magnitude calculation
    avg_token_feature = visual_tokens.mean(dim=0, keepdim=True)
    activation = (visual_tokens - avg_token_feature).norm(dim=-1)
    mag_min, mag_max = activation.min(), activation.max()
    mag_activation = (activation - mag_min) / (mag_max - mag_min)

    # frequency calculation
    freq_token = torch.fft.fft(feat).angle()
    avg_freq_token = freq_token.mean(dim=0, keepdim=True)
    activation = (freq_token - avg_freq_token).norm(dim=-1)
    freq_min, freq_max = activation.min(), activation.max()
    freq_activation = (activation - freq_min) / (freq_max - freq_min)
    return mag_activation, freq_activation
```

Filtro:

Sono stati predisposti i metodi per il calcolo e la visualizzazione delle magnitudini e delle frequenze.

Norma L2 delle feature dopo averle centrate.

```
def activations(visual_tokens):
    # visual_tokens: tensor with shape [H, W, C]
    # magnitude calculation
    avg_token_feature = visual_tokens.mean(dim=0, keepdim=True)
    activation = (visual_tokens - avg_token_feature).norm(dim=-1)
    mag_min, mag_max = activation.min(), activation.max()
    mag_activation = (activation - mag_min) / (mag_max - mag_min)

    # frequency calculation
    freq_token = torch.fft.fft(feat).angle()
    avg_freq_token = freq_token.mean(dim=0, keepdim=True)
    activation = (freq_token - avg_freq_token).norm(dim=-1)
    freq_min, freq_max = activation.min(), activation.max()
    freq_activation = (activation - freq_min) / (freq_max - freq_min)
    return mag_activation, freq_activation
```

Filtro:

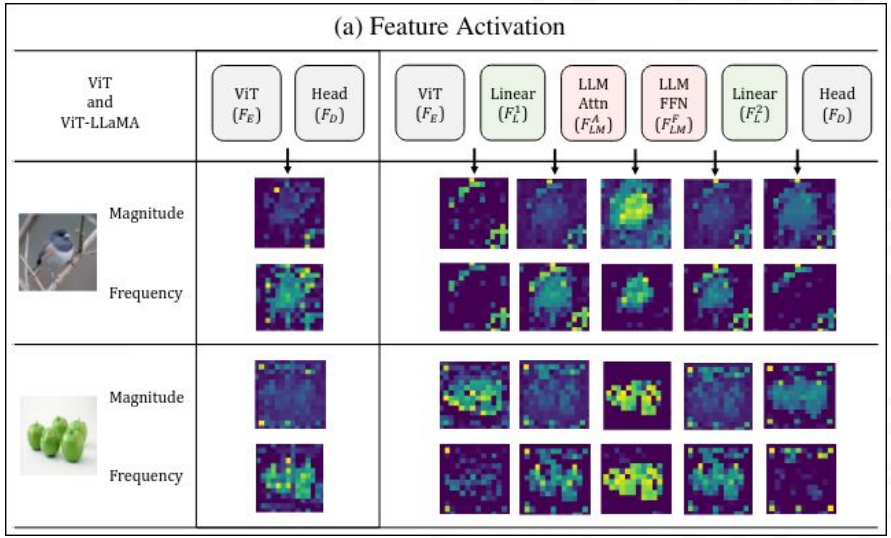
Sono stati predisposti i metodi per il calcolo e la visualizzazione delle magnitudini e delle frequenze.

Norma della differenza tra vettore delle fasi di un token vettore medio delle fasi, dopo la trasformata di Fourier.

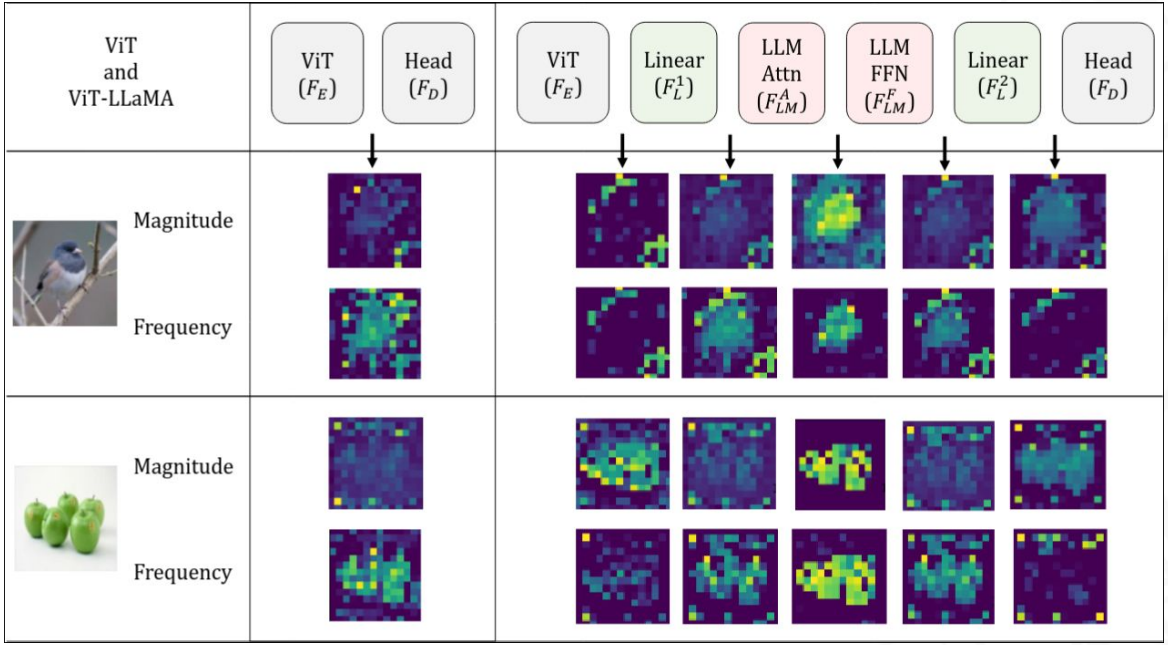
```
def activations(visual_tokens):
    # visual_tokens: tensor with shape [H, W, C]
    # magnitude calculation
    avg_token_feature = visual_tokens.mean(dim=0, keepdim=True)
    activation = (visual_tokens - avg_token_feature).norm(dim=-1)
    mag_min, mag_max = activation.min(), activation.max()
    mag_activation = (activation - mag_min) / (mag_max - mag_min)

    # frequency calculation
    freq_token = torch.fft.fft(feat).angle()
    avg_freq_token = freq_token.mean(dim=0, keepdim=True)
    activation = (freq_token - avg_freq_token).norm(dim=-1)
    freq_min, freq_max = activation.min(), activation.max()
    freq_activation = (activation - freq_min) / (freq_max - freq_min)
    return mag_activation, freq_activation
```

Filtro:

Sono state poi estratte le attivazioni delle feature dopo ogni layer.

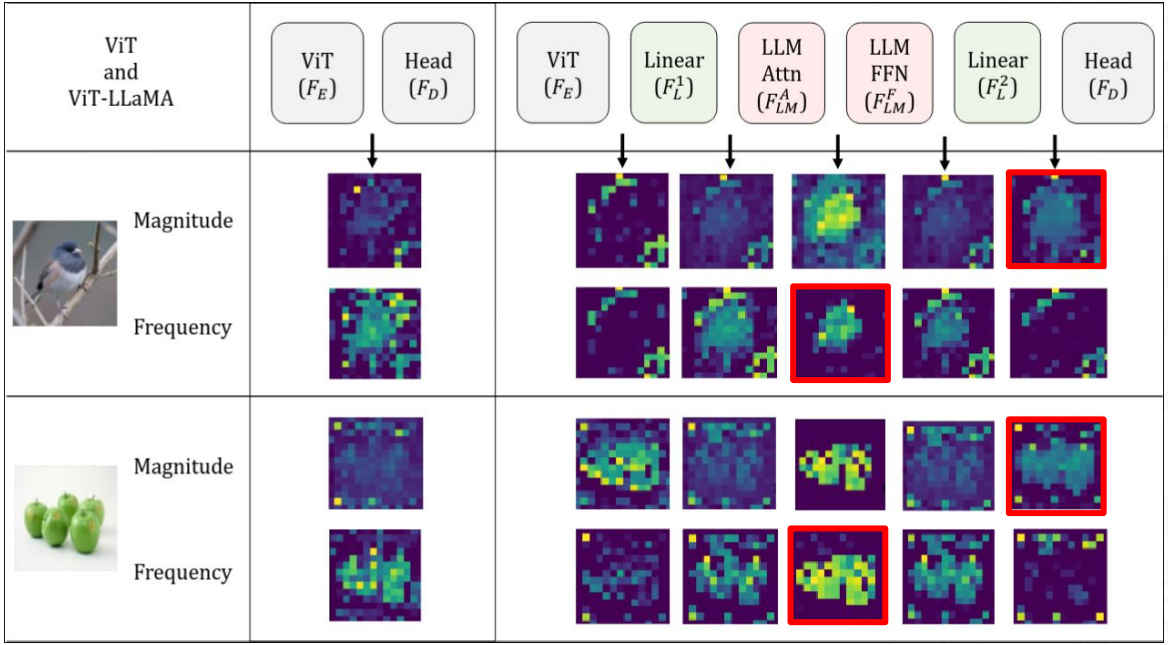


Filtro:



Vengono evidenziati i token più informativi

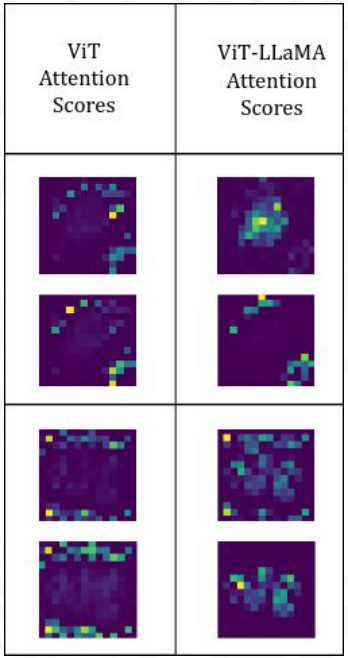
Filtro:



Si nota una tendenza alla **segmentazione**, **non comune** in ViT non progettati per tale scopo.

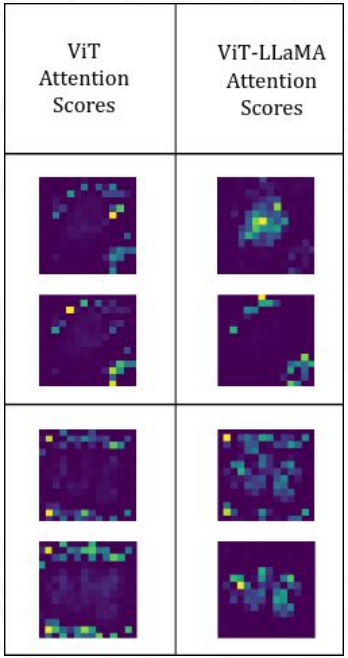
Analisi degli Attention Scores:

Si calcolano anche gli score di attenzione tra il token CLS e i visual tokens dell'ultimo blocco transformer.



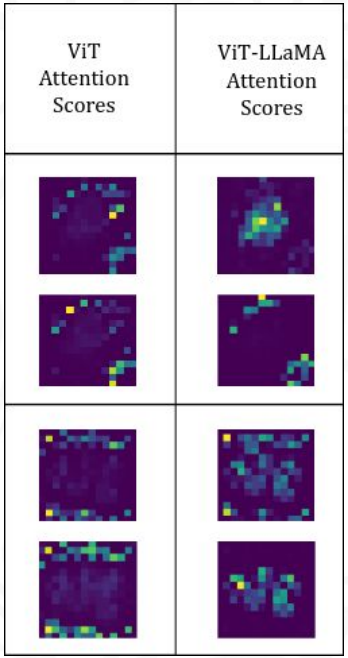
Analisi degli Attention Scores:

Degli score ideali dovrebbero mostrare segni di segmentazione.
Gli scores invece appaiono rumorosi e sparsi nella maggior parte delle teste di attenzione.



Analisi degli Attention Scores:

La capscità del modello non può dunque essere spiegata dagli scores di attenzione.



Amplificazione:

Sappiamo dall'analisi precedente che il **transformer congelato** del LLM identifica i token **informativi**.

Amplificazione:

Questi token aiuterebbero naturalmente la predizione,
ma **solo se** il decoder prendesse **tutti** i token visuali come input.

Amplificazione:

Nel nostro caso (ViT), il decoder prende in ingresso solo il token **CLS**.

Amplificazione:

Si ipotizza dunque che sia proprio il transformer congelato ad **amplificare** i contributi di tali tokens all'interno del token CLS.

Formulazione dell'ipotesi (Formale):

Formulazione del token **CLS**, ottenuto come aggregazione pesata dei visual token tramite gli **attention scores**:

$$z_L^2[\mathrm{CLS}] = \mathbf{F}_L^2 \cdot \mathbf{F}_{LM}^F \cdot \mathbf{F}_{LM}^A \left(\sum_{v \in V} w_v \, z_L^1[v] \right)$$

Formulazione dell'ipotesi (Formale):

Si scompongono i visual token tra **informativi** (soggetto) e **non-informativi** (background):

$$z_L^2[\texttt{CLS}] = \mathbf{F}_L^2 \cdot \mathbf{F}_{LM}^F \cdot \mathbf{F}_{LM}^A \left(\sum_{i \in V_i} w_i \, z_L^1[i] + \sum_{u \in V_u} w_u \, z_L^1[u] \right)$$

Troppa importanza agli scores di attenzione, troppo poca al passaggio dei layers.

Formulazione dell'ipotesi (Formale):

Token informativi, fortemente dipendenti dal passaggio attraverso i layer del transformer.

$$z_L^2[i] = \mathbf{F}_L^2 \cdot \mathbf{F}_{LM}^F \cdot \mathbf{F}_{LM}^A(z_L^1[i]), \text{ where } i \in V_i.$$

Formulazione dell'ipotesi (Formale):

Motivo per cui possiamo riformulare la precedente equazione come segue:

$$z_L^2[\texttt{CLS}] \propto \sum_{i \in V_i} w_i \underbrace{\left(\mathbf{F}_L^2 \cdot \mathbf{F}_{LM}^F \cdot \mathbf{F}_{LM}^A(z_L^1[i])\right)}_{z_L^2[i]} + \sum_{u \in V_u} w_u \underbrace{\left(\mathbf{F}_L^2 \cdot \mathbf{F}_{LM}^F \cdot \mathbf{F}_{LM}^A(z_L^1[u])\right)}_{z_L^2[u]}.$$

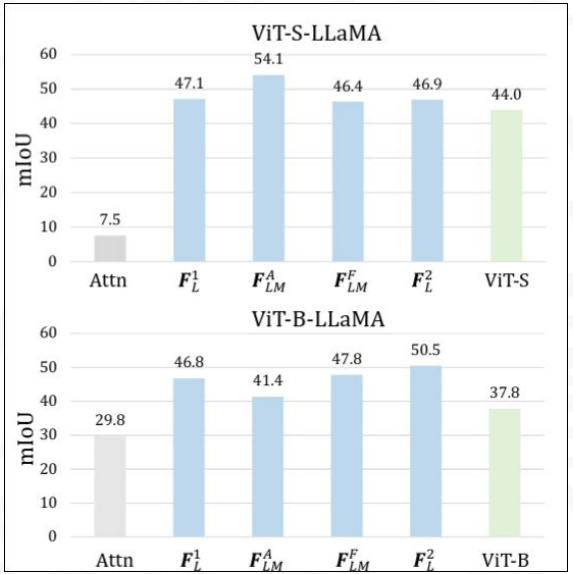
Il CLS non è l'aggregazione dei token grezzi seguita dall'azione dei layer, ma è l'azione dei layer sui token grezzi seguita dall'aggregazione.

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Evidenze Quantitative

Evidenze quantitative:

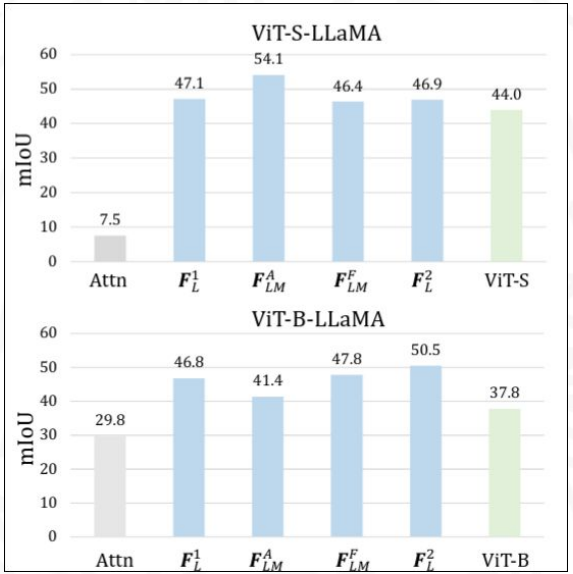
Viene utilizzato **ImageNet-S**, che fornisce **maschere di segmentazione** di **regioni informative**.



Evidenze quantitative:

Si generano **pseudo-maschere** a partire:

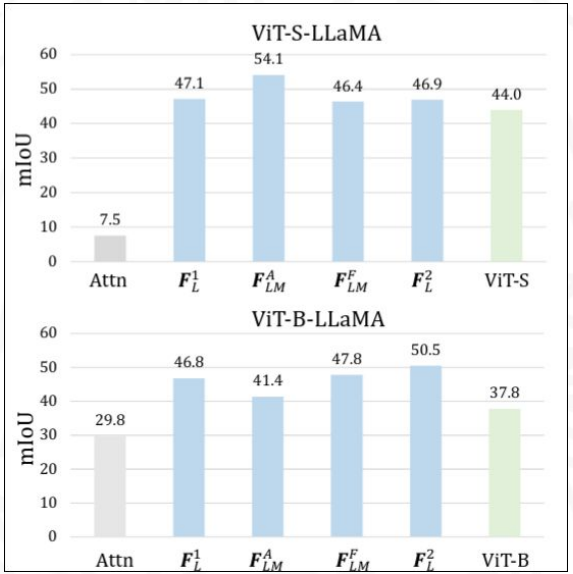
- dalle **attivazioni delle feature**
- dagli **attention scores**



Evidenze quantitative:

Si utilizza **mIoU** come indice, il quale:

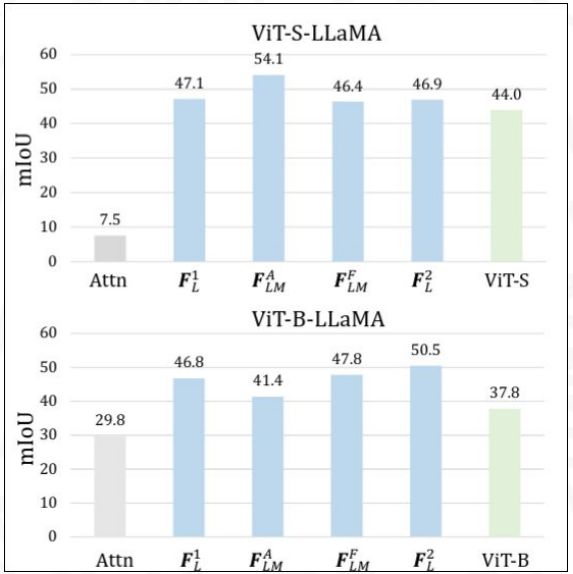
misura l'allineamento medio tra i token **altamente attivati** (o con alti attention scores) e le regioni informative ground truth.



Evidenze quantitative:

Alcune osservazioni:

- Le **feature attivate** superano gli **attention scores** in mIoU
- **ViT-LLaMA > ViT**, confermando i benefici del transformer congelato
- Effetti positivi visibili sin dai layer **iniziali**



Conclusioni:

- I blocchi transformer di **LLM congelati** possono essere efficacemente integrati come **encoder generici** migliorando le prestazioni in modo consistente.
- L'**ipotesi di information filtering** evidenzia la versatilità dei transformer degli LLM come modelli di **rappresentazione generale**.

Limitazioni:

- Gli esperimenti privilegiano **ampia copertura di task**, ma **non mirano allo stato dell'arte** per ciascun problema.
- L'ipotesi proposta **non spiega il ruolo dei singoli layer** né le **dinamiche di training** che allineano le feature visive col transformer congelato.

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Frozen Transformers in Language Models Are Effective Visual Encoder Layers

Ziqi Pang Ziyang Xie Yunze Man Yu-Xiong Wang

ICLR 2024

Cosimo Borghini